UNIDAD 5. ESTRUCTURAS DE ALMACENAMIENTO

Índice

1. Arrays Unidimensionales (Vectores) 2
   1.1 Declaración y Creación 2
   1.2 Inicialización de Arrays 4
   1.3 Acceso a elementos: Índices, límites y longitud 6
2. Algoritmos y Manipulación de Arrays 8
   2.1 Recorridos: `for` clásico vs `for-each` 8
   2.2 Búsquedas: Lineal y Binaria 10
   2.3 Ordenaciones 12
3. Arrays Multidimensionales (Matrices) 14
   3.1 Declaración y Creación: Estructura de filas y columnas 14
   3.2 Inicialización y Acceso: Sintaxis de doble índice `[][]` 16
4. Cadenas de caracteres (Clase String) 17
   4.1 Definición: Declaración y creación 17
   4.2 Acceso y Propiedades: Acceso a caracteres y longitud 19
   4.3 Operaciones básicas con Strings 21

Resumen 23

1. Arrays Unidimensionales (Vectores)

Hasta ahora, si querías guardar las notas de 3 alumnos, creabas 3 variables: `nota1`, `nota2`, `nota3`. Pero, ¿y si tienes 100 alumnos? ¿O 1.000? Sería imposible manejar 1.000 variables distintas. Aquí es donde entran los **Arrays** (también llamados arreglos, vectores o matrices).

Un Array es una estructura de datos que nos permite almacenar **múltiples valores** del **mismo tipo** bajo un **único nombre**. Piensa en un array como si fuera una hilera de taquillas numeradas, una al lado de la otra.

1.1 Declaración y Creación

En Java, los arrays son **Objetos**. Esto significa que siguen las mismas reglas de memoria (Stack vs Heap) que vimos en la unidad anterior con las clases. Crear un array es un proceso de dos pasos: definir la variable y construir el objeto.

Paso 1: Declaración (La Referencia)

Primero avisamos a Java de que vamos a necesitar una variable para "apuntar" a un array.

- **Sintaxis:** `TipoDato[] nombreVariable;`
- **¿Qué ocurre en memoria?** Se crea un espacio en el **Stack** con valor `null`. Aún no existe el contenedor de datos, solo el "mando a distancia".

Java
```
// Declaración estándar (Recomendada en Java)
int[] edades;

// Declaración estilo C (Válida, pero menos usada)
int edades[];
```

En este punto, si intentas usar `edades`, obtendrás un error porque el array no existe.

Paso 2: Creación (La Instanciación)

Aquí es donde reservamos el espacio físico real para guardar los datos.

- **Sintaxis:** `nombreVariable = new TipoDato[TAMAÑO];`
- **La Regla de Oro:** Debes indicar el **tamaño fijo** al crearlo. Una vez creado, **no puede crecer ni encoger**.
- **¿Qué ocurre en memoria?** Java busca un hueco en el **Heap** lo suficientemente grande para guardar ese número de casillas consecutivas.

Java
```
// Creamos un array de 5 huecos para enteros
edades = new int[5];
```

C. Declaración y Creación en una línea

Es lo más habitual. Combinamos ambos pasos:

Java
```
// "Crea una variable 'notas' que apunte a un nuevo array de 10 doubles"
double[] notas = new double[10];
```

D. Diferencia entre Referencia y Objeto (Modelo de Memoria)

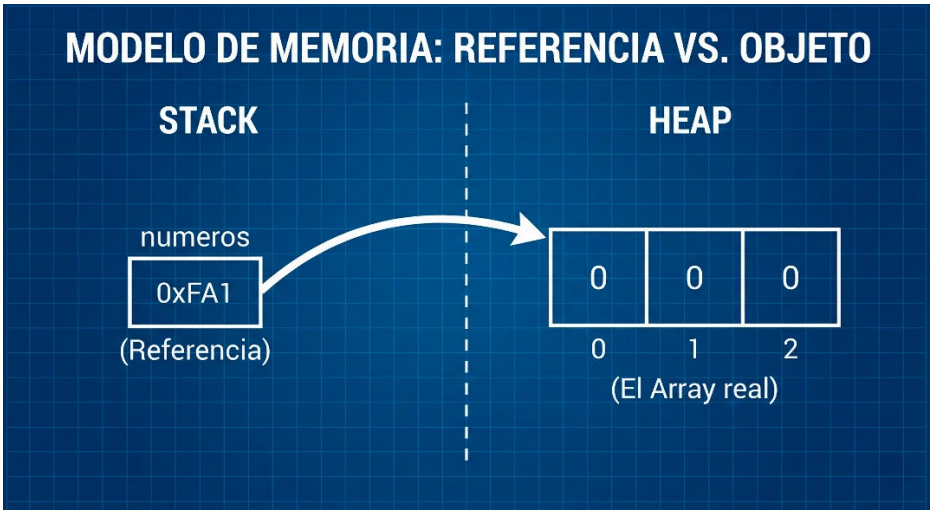
Es vital entender el dibujo mental de lo que acabamos de escribir. Supongamos: `int[] numeros = new int[3];`

1. **La Referencia (`numeros`):**

   - Vive en el **Stack**.
   - Contiene una dirección de memoria (ej: `0xFA1`).
   - Es pequeña y ligera.

2. **El Objeto Array (`new int[3]`):**

   - Vive en el **Heap**.
   - Es un bloque contiguo de memoria dividido en 3 celdas.
   - Java inicializa automáticamente esas celdas con el valor por defecto (`0` para int).



Consecuencias de esto: Si haces `int[] lista2 = numeros;`, **NO** estás copiando los datos. Estás creando otra referencia (`lista2`) que apunta al **mismo** array del Heap. Si cambias un dato usando `lista2`, también cambia para `numeros`.

Identidad: Un array es un objeto especial que sabe su propio tamaño.

1.2 Inicialización de Arrays

A. Inicialización por Defecto (Automática)

Si declaras y creas un array pero no guardas nada en él, Java no lo deja vacío. El sistema rellena automáticamente todas las posiciones con el **valor por defecto** correspondiente al tipo de dato.

Esto ocurre en el momento exacto de ejecutar el `new`.

Tipo de Dato	Se rellena con...
`int`, `short`, `byte`, `long`	`0`
`double`, `float`	`0.0`
`boolean`	`false`
`char`	`\u0000` (Carácter nulo)
Objetos (`String`, `Persona`...)	**`null`** (¡Cuidado aquí!)

Ejemplo:
Java
```
// Creamos un array de 3 Strings
String[] nombres = new String[3];

// ¿Qué hay dentro?
// [null, null, null]
// Si intentas hacer nombres[0].toUpperCase() el programa fallará
(NullPointerException).
```

B. Inicialización Explícita (Estática / Inline)

Se utiliza cuando sabemos de antemano qué valores queremos guardar en el momento de escribir el código. Java nos permite usar una sintaxis abreviada con llaves {}.

- **Sintaxis:** `Tipo[] nombre = { valor1, valor2, valor3 };`
- **Características:**

1. No usas la palabra `new` (Java la hace implícitamente).
2. No especificas el tamaño (Java cuenta los elementos entre las llaves y fija el tamaño automáticamente).

Ejemplo:

Java
```
// Creamos un array de tamaño 5 con las vocales
char[] vocales = { 'a', 'e', 'i', 'o', 'u' };

// Creamos un array de enteros
int[] primos = { 2, 3, 5, 7, 11 };
```

Importante: Esta sintaxis abreviada solo funciona si declaras e inicializas **en la misma línea**.

Java
```
int[] n;
n = { 1, 2, 3 }; // ❌ ERROR DE COMPILACIÓN
```

C. Inicialización Dinámica (Asignación manual)

Es la forma más común en programación real. Se usa cuando los datos no se conocen al principio (vienen de una base de datos, los escribe el usuario por teclado o son fruto de un cálculo).

Asignamos valores posición a posición usando el índice (posición).

1. Asignación directa:

Java
```
int[] puntuaciones = new int[3]; // [0, 0, 0]

puntuaciones[0] = 50;  // Ponemos un 50 en la primera caja
puntuaciones[1] = 90;
puntuaciones[2] = 10;
// Estado final: [50, 90, 10]
```

2. Asignación mediante Bucle (Dinámica real): Usamos un bucle `for` para rellenar el array automáticamente o pidiendo datos al usuario.

Java
```
import java.util.Scanner;

public class EjemploDinamico {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        double[] precios = new double[5];

        // Rellenamos pidiendo al usuario (Entrada)
        for (int i = 0; i < precios.length; i++) {
            System.out.print("Introduce precio " + (i+1) + ": ");
            precios[i] = teclado.nextDouble();
        }

        // Ahora el array contiene los 5 datos que escribió el
usuario.
    }
}
```

Resumen de Estrategia:

- ¿El array empieza vacío o con ceros? -> Usa `new` (Por defecto).
- ¿Sabes los valores fijos (días de la semana)? -> Usa `{}` (Explícita).
- ¿Los datos los decide el usuario o un cálculo? -> Usa `new` **+ Bucle** (Dinámica).

1.3 Acceso a elementos: Índices, límites y longitud

A. El uso de Índices (0-based indexing)

Para acceder a una "taquilla" individual del array, usamos los corchetes `[]` con un número entero dentro. Ese número es el **Índice**.

La Regla de Oro del Índice: En Java (y en casi todos los lenguajes de programación), **se empieza a contar desde 0**.

- El primer elemento está en la posición **0**.
- El segundo elemento está en la posición **1**.
- El último elemento está en la posición **N - 1** (donde N es el tamaño total).

Sintaxis de acceso: Podemos usar el acceso para dos cosas: leer o escribir.

Java
```
int[] numeros = { 10, 20, 30, 40, 50 }; // Tamaño 5

// 1. LECTURA (Leer lo que hay en la posición 0)
int primerNumero = numeros[0]; // Devuelve 10
System.out.println(primerNumero);

// 2. ESCRITURA (Machacar lo que hay en la posición 2)
numeros[2] = 99; // El 30 desaparece. Ahora hay un 99.

// 3. USO DIRECTO EN EXPRESIONES
// Sumamos el contenido de la casilla 1 (20) y la 4 (50)
int suma = numeros[1] + numeros[4]; // 70
```

B. La propiedad `.length` (Longitud)

Los arrays en Java son "listos". Saben cuánto miden. Todos los arrays tienen una propiedad pública final llamada .length que contiene el tamaño con el que fue creado.

- **Nota Importante:** Es una propiedad, no un método.
  - `array.length` ✅ (Correcto en Arrays)
  - `array.length()` ❌ (Error: Esto es para Strings)
  - `array.size()` ❌ (Error: Esto es para Listas)

Esta propiedad es fundamental para evitar errores y para recorrer el array sin saber de antemano su tamaño.

Java
```
String[] dias = { "Lunes", "Martes", "Miércoles" };
```

```
System.out.println("El array tiene " + dias.length + "
elementos.");
// Imprime: 3

// TRUCO PROFESIONAL: ¿Cómo acceder al último elemento sin saber el
número?
// Como empieza en 0, el último siempre es "longitud menos 1".
String ultimoDia = dias[dias.length - 1]; // dias[2] -> "Miércoles"
```

C. Límites y Errores (`ArrayIndexOutOfBoundsException`)

Este es el error más famoso que vas a encontrar trabajando con arrays. Ocurre cuando intentas acceder a una taquilla que **no existe**.

El rango válido de índices va siempre de `0` a **`length - 1`**.

Si intentas acceder fuera de ese rango (por arriba o por abajo), Java no te devuelve basura ni un error silencioso. Lanza una **Excepción** (un error fatal que detiene el programa inmediatamente).

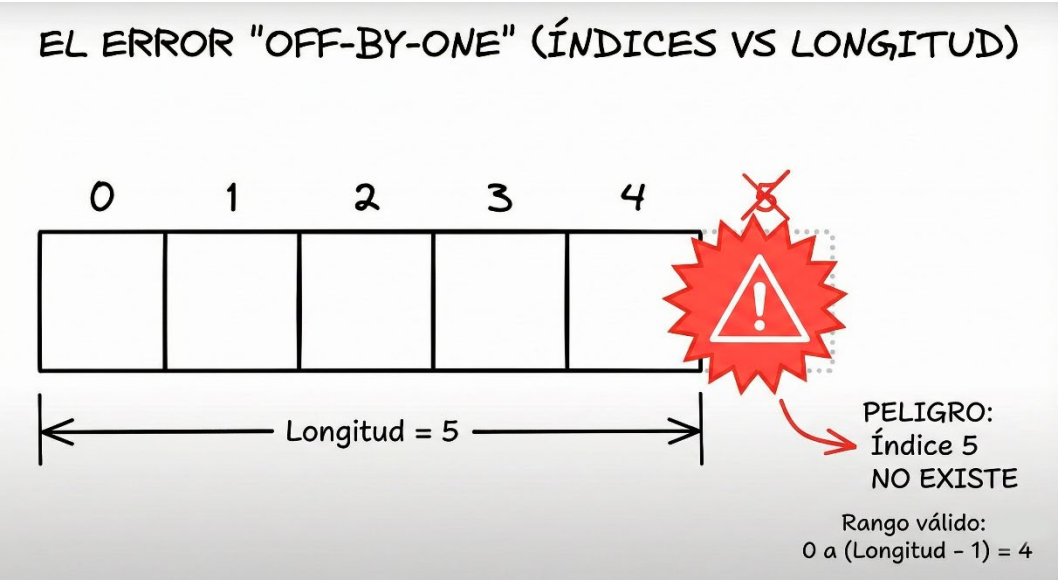
El error clásico "Off-by-one" (Por uno):

Java
```
int[] datos = new int[5]; // Índices válidos: 0, 1, 2, 3, 4

// CASO 1: Índice negativo
// datos[-1] = 5; // ❌ ERROR FATAL:
ArrayIndexOutOfBoundsException: -1

// CASO 2: Índice igual al tamaño (El error más común)
// datos[5] = 10; // ❌ ERROR FATAL:
ArrayIndexOutOfBoundsException: 5
```

¿Por qué falla `datos[5]` si el tamaño es 5? Porque al empezar en 0, la casilla 5 sería la sexta casilla, y solo hemos reservado 5.



2. Algoritmos y Manipulación de Arrays

2.1 Recorridos: `for` clásico vs `for-each`

Recorrer (o iterar) un array significa visitar **todas** sus casillas, desde la primera hasta la última, para realizar una operación común (imprimirlas, sumarlas, buscar un valor, etc.).

En Java disponemos de dos herramientas principales para esto.

A. El Bucle `for` Clásico (Acceso por Índice)

Es la forma tradicional y la más flexible. Se basa en usar una variable contadora (usualmente `i`) que actúa como índice para acceder a cada posición mediante `array[i]`.

- **Sintaxis:** `for (int i = 0; i < array.length; i++) { ... }`
- **Clave:** Usamos la propiedad `.length` en la condición para que el bucle se adapte automáticamente al tamaño del array.

Uso 1: Lectura (Consultar datos)

Java
```
int[] numeros = { 10, 20, 30, 40 };

// "Desde la casilla 0 hasta la última, de una en una"
for (int i = 0; i < numeros.length; i++) {
    System.out.println("En la posición " + i + " hay un: " +
numeros[i]);
}
```

Uso 2: Escritura (Modificar el array). Como tenemos el índice `i`, podemos escribir dentro de las casillas.

Java
```
// Vamos a duplicar el valor de todos los números
for (int i = 0; i < numeros.length; i++) {
    numeros[i] = numeros[i] * 2; // Acceso de escritura
}
// Ahora el array es { 20, 40, 60, 80 }
```

Uso 3: Recorridos complejos: El `for` clásico permite recorrer al revés, de dos en dos, o solo la mitad.

Java
```
// Recorrido inverso (Del final al principio)
for (int i = numeros.length - 1; i >= 0; i--) {
    System.out.println(numeros[i]);
}
```

B. El Bucle `for-each` (Acceso por Elemento)

Introducido en Java 5, es una versión simplificada y más legible diseñada específicamente para recorrer colecciones y arrays.

- **Lectura:** "Para cada **elemento** dentro del **array**..."
- **Sintaxis:** `for (Tipo variableTemporal : nombreArray) { ... }`
- **Funcionamiento:** En cada vuelta, Java copia automáticamente el contenido de la siguiente casilla en la `variableTemporal`.

Java
```
String[] frutas = { "Manzana", "Pera", "Plátano" };

// FOR-EACH: Mucho más limpio
for (String fruta : frutas) {
    System.out.println("Me gusta la " + fruta);
}
```

Limitación Importante (Solo Lectura): En el `for-each`, la variable (`fruta`) es una copia temporal del valor. Si modificas `fruta`, NO se modifica el array original.

Java
```
for (int n : numeros) {
    n = 0; // Esto solo pone a 0 la variable temporal 'n'.
}
// El array 'numeros' sigue intacto. No se ha borrado nada.
```

C. Tabla Comparativa: ¿Cuál elijo?

Característica	For Clásico (for int i...)	For-Each (for Tipo e...)
Sintaxis	Más verbosa, hay que gestionar índices.	**Limpia y directa**.
Acceso	Tienes el **Índice** (`i`) y el **Valor** (`a[i]`).	Solo tienes el **Valor**. No sabes en qué posición estás.
Modificación	**Sí**. Puedes cambiar el contenido del array (`a[i] = x`).	**No**. Es de solo lectura para el array.
Dirección	Flexible (Adelante, atrás, saltos...).	Rígida (Siempre del primero al último, de 1 en 1).
Seguridad	Riesgo de *OutOfBounds* si fallas en la condición.	**Seguro 100%**. Nunca se sale del rango.

Regla de decisión:

1. ¿Necesitas **modificar** los datos del array? -> Usa **For Clásico**.
2. ¿Necesitas saber la **posición** (índice) del dato? -> Usa **For Clásico**.
3. ¿Solo quieres **leer/mostrar/sumar** todos los datos en orden? -> Usa **For-Each** (Es más elegante).

2.2 Búsquedas: Lineal y Binaria

A. Búsqueda Lineal (o Secuencial)

Es el método intuitivo, el que haría cualquier persona si le das una baraja de cartas desordenada y le pides buscar el As de Copas.

- **Algoritmo:** Empiezas en la primera posición (0) y vas mirando una a una hasta que encuentras lo que buscas o llegas al final.
- **Requisito:** Ninguno. Funciona con arrays **desordenados** y ordenados.
- **Eficiencia:** Baja. Si tienes 1 millón de datos y el que buscas es el último, tendrás que hacer 1 millón de comparaciones.

Implementación en Java:
Java
```
public static int busquedaLineal(int[] array, int buscado) {
    // Recorremos todo el array
    for (int i = 0; i < array.length; i++) {
        // Si lo encontramos, devolvemos el índice (la posición)
        if (array[i] == buscado) {
            return i;
        }
    }
    // Si terminamos el bucle y no lo hemos encontrado, devolvemos
-1
    // (-1 es el estándar universal para decir "no existe")
    return -1;
}
```

B. Búsqueda Binaria (o Dicotómica)

Este es el algoritmo inteligente. Es el método que usas para buscar una palabra en un diccionario o un número en una guía telefónica. No empiezas por la primera página; abres por la mitad.

- **Requisito ABSOLUTO:** El array **DEBE ESTAR ORDENADO** previamente. Si los datos están desordenados, la búsqueda binaria falla y no sirve.
- **Algoritmo (Divide y Vencerás):**
  1. Miras el elemento del **centro** del array.
  2. ¿Es el que buscas? ¡Genial, fin!
  3. ¿El que buscas es **menor**? Entonces sabes que está en la mitad izquierda. Descartas toda la mitad derecha de golpe.
  4. ¿El que buscas es **mayor**? Descartas la mitad izquierda y buscas en la derecha.
  5. Repites el proceso con la mitad restante.
- **Eficiencia:** Brutalmente alta.
  - En 1.000.000 de datos, la Búsqueda Lineal haría hasta 1.000.000 de preguntas.
  - La Búsqueda Binaria haría, como máximo, **20 preguntas**.

Implementación en Java:

Java

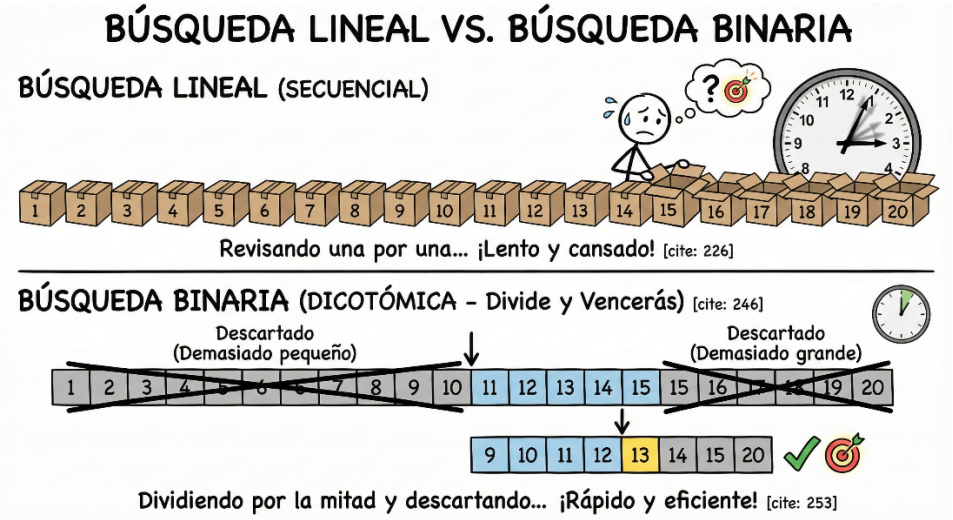
```
public static int busquedaBinaria(int[] array, int buscado) {
    int inicio = 0;
    int fin = array.length - 1;

    while (inicio <= fin) {
        // Calculamos la mitad
        int centro = (inicio + fin) / 2;

        if (array[centro] == buscado) {
            return centro; // ¡Encontrado!
        } else if (buscado < array[centro]) {
            fin = centro - 1; // Nos quedamos con la parte
izquierda
        } else {
            inicio = centro + 1; // Nos quedamos con la parte
derecha
        }
    }
    return -1; // No encontrado
}
```

C. Comparativa: ¿Cuál usar?

Característica	Búsqueda Lineal	Búsqueda Binaria
Orden de datos	Indiferente (funciona siempre).	**OBLIGATORIO** estar ordenado.
Simplicidad	Muy fácil de programar.	Lógica un poco más compleja.
Velocidad (Pocos datos)	Rápida.	Rápida.
Velocidad (Muchos datos)	Lenta (N pasos).	Instantánea ($\log N$ pasos).



Más info: Java ya trae la búsqueda binaria programada en su librería de utilidades.

Java
```
import java.util.Arrays;
```

int[] numeros = { 1, 3, 5, 7, 9 }; // Recuerda: Debe estar ordenado int posicion = Arrays.binarySearch(numeros, 5); // Devuelve 2

2.3 Ordenaciones

Existen decenas de algoritmos de ordenación (Burbuja, Inserción, Selección, QuickSort, MergeSort...). Como programador, debes distinguir dos enfoques: el **académico** (entender cómo funciona la lógica de mover datos) y el **profesional** (usar las herramientas optimizadas de Java).

A. El concepto clave: El Intercambio (Swap)

Casi todos los algoritmos de ordenación se basan en comparar dos elementos y, si están desordenados, intercambiarlos de posición.

Para intercambiar el valor de dos variables (`A` y `B`), **no puedes hacerlo directamente**. Necesitas una **variable auxiliar** (temporal).

> **Analogía de los vasos:** Tienes un vaso con Fanta y otro con Coca-Cola. Para intercambiar los líquidos sin mezclarlos, necesitas un tercer vaso vacío.

Java
```
int a = 5;
int b = 10;

// INTERCAMBIO INCORRECTO
// a = b; // a vale 10
// b = a; // b vale 10 (perdimos el 5 para siempre)
```

```
// INTERCAMBIO CORRECTO (Swap)
int aux = a; // Guardamos el 5 en el vaso auxiliar
a = b;       // a ahora vale 10
b = aux;     // b recupera el 5 del auxiliar
```

B. Algoritmo Académico: La Burbuja (Bubble Sort)

Es el algoritmo más famoso para aprender, aunque es poco eficiente para grandes cantidades de datos. Su lógica es sencilla: Compara parejas de elementos adyacentes. Si el de la izquierda es mayor que el de la derecha, los intercambia. Repite este proceso varias veces hasta que el número más grande "burbujea" hasta la posición final.

Implementación: Requiere dos bucles anidados (uno dentro de otro).

Java
```
int[] numeros = { 5, 1, 4, 2, 8 };

// Bucle externo: Repite el proceso tantas veces como elementos
haya
for (int i = 0; i < numeros.length - 1; i++) {

    // Bucle interno: Recorre el array comparando parejas
    // (El -i es una optimización: no miramos los últimos que ya
están ordenados)
    for (int j = 0; j < numeros.length - 1 - i; j++) {

        // Si el actual es MAYOR que el siguiente...
        if (numeros[j] > numeros[j + 1]) {
            // ...los intercambiamos (Swap)
            int aux = numeros[j];
            numeros[j] = numeros[j + 1];
            numeros[j + 1] = aux;
        }
    }
}
// Resultado: { 1, 2, 4, 5, 8 }
```

C. El enfoque Profesional: `Arrays.sort()`

En la vida real, rara vez escribirás un algoritmo de burbuja (es lento y propenso a errores). Java incluye en su librería estándar algoritmos de ordenación de altísimo rendimiento (generalmente una variante de QuickSort para primitivos y TimSort para objetos).

Para usarlo, necesitamos la clase de utilidades `java.util.Arrays`.

Java
```
import java.util.Arrays;

public class OrdenacionPro {
    public static void main(String[] args) {
        int[] notas = { 9, 2, 5, 1, 7 };

        // Una sola línea. Rápido, optimizado y sin errores.
        Arrays.sort(notas);
```

```
        // Imprimimos para verificar
        // Arrays.toString es un truco para ver el array sin hacer
un bucle for
        System.out.println(Arrays.toString(notas));
        // Salida: [1, 2, 5, 7, 9]
    }
}
```

Más info: `Arrays.sort()` modifica el array **original**. Es una operación destructiva (pierdes el orden que tenías antes).

3. Arrays Multidimensionales (Matrices)

3.1 Declaración y Creación: Estructura de filas y columnas

Técnicamente, Java no tiene "matrices" reales como otros lenguajes. Lo que tiene son **Arrays de Arrays**. Es decir, un array principal donde cada casilla, en lugar de guardar un número, guarda una referencia a otro array.

Sin embargo, para facilitarnos la vida, las visualizamos mentalmente como una **tabla con filas y columnas**.

A. Sintaxis de Declaración

Para indicar que queremos dos dimensiones, simplemente añadimos un segundo par de corchetes `[][]`.

- **1 Dimensión:** `int[] vector;`
- **2 Dimensiones:** `int[][] matriz;`
- **3 Dimensiones:** `int[][][] cubo;` (y así sucesivamente, aunque rara vez pasarás de 2).

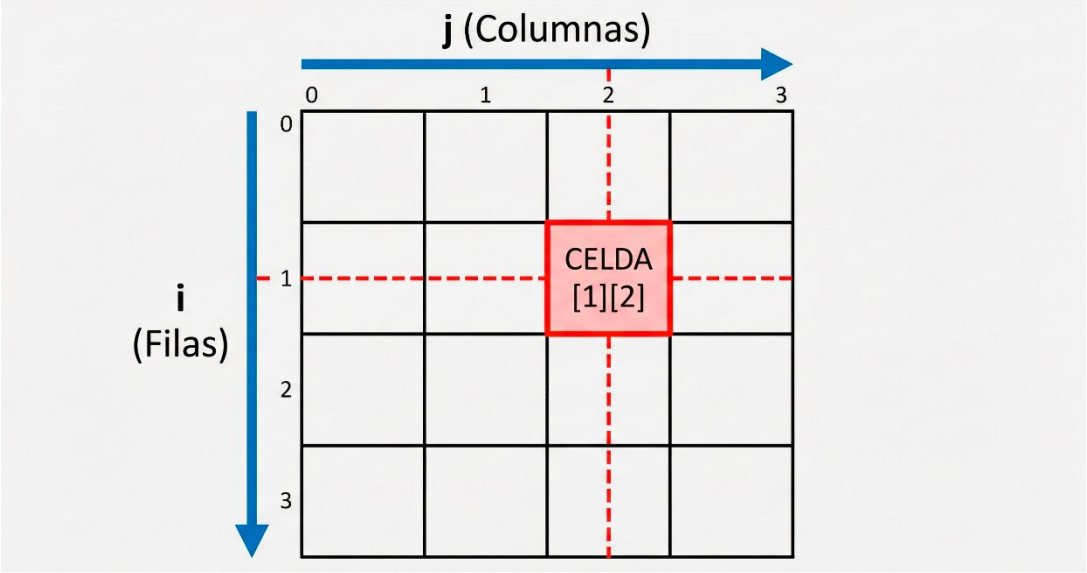
B. Creación (Instanciación)

Al hacer el `new`, debemos decir el tamaño de ambas dimensiones.

- **Sintaxis:** `variable = new Tipo[FILAS][COLUMNAS];`

Regla Mnemotécnica (RC): El primer corchete siempre indica las Rows (Filas / Vertical). El segundo corchete siempre indica las Columns (Columnas / Horizontal).

Ejemplo Práctico: Queremos crear una sala de cine con 3 filas y 4 asientos por fila.



Java
```
// Declaración y Creación
int[][] cine = new int[3][4];

/*
VISUALMENTE:
        [0] [1] [2] [3]   <-- Columnas
     [0]  0   0   0   0
     [1]  0   0   0   0
     [2]  0   0   0   0
      ^
    Filas
*/
```

C. La realidad en Memoria (Array de Arrays)

Es importante entender esto para no confundirse después con el `.length`.

Como dijimos, Java crea un array principal (las filas). Cada posición de ese array contiene un "mando a distancia" que apunta a otro array independiente (las columnas).

Esto implica dos cosas sobre el tamaño (`.length`):

1. **`matriz.length`**: Te dice cuántas **FILAS** hay (el tamaño del array principal vertical).
2. **`matriz[0].length`**: Entras en la primera fila y preguntas su tamaño. Esto te dice cuántas **COLUMNAS** tiene esa fila.

Java
```
int[][] tabla = new int[5][10]; // 5 filas, 10 columnas

System.out.println("Número de filas: " + tabla.length);        // 5
System.out.println("Número de columnas: " + tabla[0].length); // 10
```

En definitiva:

- **Declarar:** Usa `[][]`.
- **Crear:** `new int[FILAS][COLUMNAS]`.
- **Estructura:** Piensa siempre en coordenadas `(y, x)` o `(fila, columna)`.

3.2 Inicialización y Acceso: Sintaxis de doble índice `[][]`

A. Inicialización (Rellenar la matriz)

Al igual que con los arrays simples, tenemos dos formas de dar valores iniciales.

1. Inicialización Explícita (Estática). Si sabemos los datos desde el principio, podemos dibujarlos visualmente usando llaves anidadas `{ {..}, {..} }`. Java detectará automáticamente el número de filas y columnas.

Java

```
int[][] tablero = {
    { 1, 2, 3 },    // Fila 0
    { 4, 5, 6 },    // Fila 1
    { 7, 8, 9 }     // Fila 2
};
// Resultado: Una matriz de 3 filas x 3 columnas.
```

2. Inicialización Dinámica (Celda a celda). Si hemos usado `new`, la matriz está llena de ceros. Asignamos valores usando las coordenadas.

Java

```
int[][] cine = new int[3][3];

cine[0][0] = 1;  // Arriba a la izquierda
cine[1][1] = 1;  // Centro
cine[2][2] = 1;  // Abajo a la derecha (Diagonal)
```

B. Acceso a Elementos (Coordenadas)

Para acceder, usamos dos pares de corchetes: `matriz[FILA][COLUMNA]`.

Piensa en el juego de "Hundir la flota" o en una hoja de Excel:

- Para leer: `int x = matriz[1][2];` (Fila 1, Columna 2).
- Para escribir: `matriz[1][2] = 99;`

¡Cuidado con los índices! Recuerda que todo empieza en 0. En una matriz de 3x3, la última celda no es la `[3][3]`, sino la `[2][2]`.

C. Recorridos: Bucles Anidados

Para procesar una matriz entera (por ejemplo, para poner todos los contadores a cero o para buscar un dato), un solo bucle `for` no es suficiente, porque solo recorrería las filas.

Necesitamos un **Bucle Anidado** (un `for` dentro de otro `for`):

1. **Bucle Externo (`i`):** Controla la **Fila** actual.
2. **Bucle Interno (`j`):** Recorre todas las **Columnas** de esa fila.

La estructura estándar:
Java
```
int[][] matriz = new int[3][4];

// Bucle EXTERNO: Recorre las FILAS (de arriba a abajo)
// Usamos matriz.length (Total de filas)
for (int i = 0; i < matriz.length; i++) {

    // Bucle INTERNO: Recorre las COLUMNAS (de izquierda a derecha)
    // Usamos matriz[i].length (Tamaño de la fila actual)
    for (int j = 0; j < matriz[i].length; j++) {

        // AQUÍ ACCEDEMOS A LA CELDA INDIVIDUAL
        matriz[i][j] = 5; // Ponemos un 5 en todas las celdas
        System.out.print(matriz[i][j] + " "); // Imprimimos
    }

    // Al terminar una fila entera, hacemos un salto de línea
    System.out.println();
}
```

Visualización de la ejecución: El código funciona así:

1. Fija Fila `i=0`. Mueve `j`: `[0][0]`, `[0][1]`, `[0][2]`... (Termina la fila).
2. Salta de línea.
3. Fija Fila `i=1`. Mueve `j`: `[1][0]`, `[1][1]`, `[1][2]`... (Termina la fila).
4. Repite hasta acabar.

4. Cadenas de caracteres (Clase String)

4.1 Definición: Declaración y creación

La clase **`String`** (con "S" mayúscula, porque es una Clase, no un tipo primitivo) representa una secuencia inmutable de caracteres.

A. Declaración

Como cualquier objeto, primero declaramos la variable (la referencia).

Java
```
String saludo;
// En este punto, 'saludo' vale null.
// No apunta a ningún texto, ni siquiera a un texto vacío.
```

B. Creación e Inicialización: El duelo (Literal vs `new`)

Aquí es donde Java es especial. A diferencia de otros objetos donde siempre usamos `new`, con los Strings tenemos dos formas de crearlos, y la diferencia afecta al rendimiento y a la memoria.

1. Mediante Literales (La forma recomendada). Un **literal** es cualquier texto escrito entre comillas dobles "...". Cuando usas esta forma, Java hace magia para ahorrar memoria.

Java
```
String a = "Hola";
```

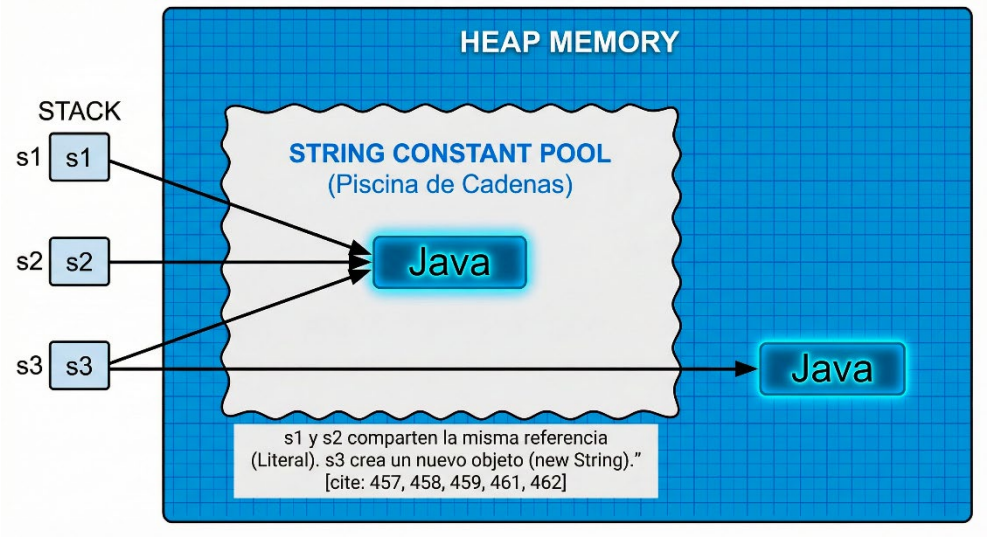
2. Mediante el operador `new` (La forma "fuerza bruta"). Como `String` es una clase, puedes llamar a su constructor. Esto obliga a Java a crear un objeto nuevo sí o sí.

Java
```
String b = new String("Hola");
```

C. Gestión de Memoria: El "String Pool"

¿Por qué existen dos formas? Por el **String Constant Pool** (Piscina de constantes de cadena). El Pool es una zona especial dentro del Heap donde Java guarda los textos para reutilizarlos.

1. **Cuando usas Literales (`"Hola"`):** Java mira en el Pool.
    - ¿Ya existe "Hola" allí? -> **Reutiliza** la referencia. No crea memoria nueva.
    - ¿No existe? -> Lo crea en el Pool.
2. **Cuando usas `new String("Hola")`: Java ignora el Pool.**
    - Crea un objeto nuevo en el Heap normal, aunque el texto sea idéntico a uno que ya existe.



Ejemplo de la diferencia:
Java
```
// CASO 1: Literales (Optimización)
String s1 = "Java";
String s2 = "Java";
```

```
// Aunque parecen dos variables, apuntan al MISMO sitio físico en
el Pool.
// (s1 == s2) es TRUE.

// CASO 2: Operador new (Sin optimización)
String s3 = new String("Java");
String s4 = new String("Java");

// Has forzado la creación de dos objetos distintos en el Heap.
// (s3 == s4) es FALSE (Tienen direcciones de memoria distintas).
```

Consejo Profesional: En el 99% de los casos, usa siempre literales (`String s = "Texto";`). Es más corto, más legible y mucho más eficiente para la memoria RAM.

D. String Vacío vs Null

Es un error muy común confundir estas dos situaciones:

1. `String s = "";` **(Cadena Vacía):**
   - El objeto **existe**.
   - Tiene longitud 0.
   - Es como una caja vacía. Puedes usarla.
2. `String s = null;` **(Referencia Nula):**
   - El objeto **no existe**.
   - Es como no tener caja.
   - Si intentas medirlo (`s.length()`), el programa explota (`NullPointerException`).

4.2 Acceso y Propiedades: Acceso a caracteres y longitud

Para interactuar con el contenido de un texto, Java nos obliga a usar **métodos** (funciones con paréntesis), en lugar de acceder directamente a propiedades o usar corchetes `[]`.

A. La Longitud (`length()`)

Para saber cuántas letras (caracteres) tiene una cadena, usamos el método `.length()`.

La Trampa Clásica de Examen:

- En los **Arrays**, la longitud es una propiedad: `.length` (sin paréntesis).
- En los **Strings**, la longitud es un método: `.length()` **(con paréntesis)**.

Java
```
String mensaje = "Hola Mundo";

// int tamano = mensaje.length;    // ❌ ERROR DE COMPILACIÓN
int tamano = mensaje.length();     // ✅ CORRECTO. Devuelve 10
(cuenta el espacio)
```

B. Acceso a Caracteres (`charAt()`)

A diferencia de C++ o C#, en Java **NO** puedes usar corchetes para leer una letra.

- `char c = mensaje[0];` ✕ **Error**.

Debes usar el método `.charAt(int indice)`. Los índices siguen la misma regla que los arrays: empiezan en **0** y terminan en **longitud - 1**.

Java
```
String palabra = "Java";

// Quiero la primera letra ('J')
char primera = palabra.charAt(0);

// Quiero la tercera letra ('v')
char tercera = palabra.charAt(2);

// Quiero la última letra ('a')
// Usamos la lógica de (longitud - 1)
char ultima = palabra.charAt(palabra.length() - 1);
```

C. Límites y Errores (`StringIndexOutOfBoundsException`)

Si intentas pedir una letra que está fuera del rango del texto, Java lanzará una excepción específica para cadenas.

Java
```
String s = "Hola"; // Índices válidos: 0, 1, 2, 3

// char c = s.charAt(4); // ✕ ERROR:
StringIndexOutOfBoundsException
// char x = s.charAt(-1); // ✕ ERROR
```

D. Ejemplo Práctico: Recorrer una Cadena

Combinando `.length()` y `.charAt()`, podemos procesar un texto letra a letra usando un bucle.

Ejemplo: Contar cuántas letras 'a' tiene una palabra.
Java
```
String texto = "Banana";
int contador = 0;

// Recorremos desde 0 hasta la última posición
for (int i = 0; i < texto.length(); i++) {

    // Extraemos la letra actual
    char letraActual = texto.charAt(i);

    // Comparamos
    if (letraActual == 'a') {
```

```
        contador++;
    }
}

System.out.println("Hay " + contador + " aes.");
```

Diferencias Array vs String:

Operación	Array (int[])	String (String)
Saber tamaño	`arr.length` (Propiedad)	`str.length()` (Método)
Leer elemento	`arr[0]` (Corchetes)	`str.charAt(0)` (Método)
Modificar	`arr[0] = 5;` (Posible)	**IMPOSIBLE** (Inmutable)

Más info: Fíjate que no existe un método `setCharAt`. Una vez creado un String, **no se puede cambiar** una letra por otra. Solo puedes crear un String nuevo con el cambio.

4.3 Operaciones básicas con Strings

A. Concatenación: Pegando textos

Unir dos cadenas para formar una nueva se llama **concatenar**. En Java tenemos dos formas principales:

1. El Operador `+` **(Sintaxis amigable).** Es la forma más común. Java es lo suficientemente listo como para convertir automáticamente otros tipos de datos a texto si detecta un String en la suma.

Java
```
String nombre = "Juan";
int edad = 25;

// Java convierte el int 25 a "25" y lo pega todo
String frase = "Soy " + nombre + " y tengo " + edad + " años.";
// Resultado: "Soy Juan y tengo 25 años."
```

2. El método `.concat()` **(Sintaxis estricta).** Es el método purista de la clase String. Hace lo mismo, pero solo acepta otros Strings. No puedes pasarle un número directamente.

Java
```
String a = "Hola ";
String b = "Mundo";
String c = a.concat(b); // "Hola Mundo"
```

Importante: Cuando concatenas, NO modificas el String original. Java crea un TERCER objeto en memoria con el resultado. `a + b` no cambia `a`, crea un nuevo String resultante.

B. Conversiones de Tipos (Parsing)

Este es el puente entre las matemáticas y el texto.

- Si el usuario escribe "100" en un formulario, Java lo recibe como `String`. No puedes sumarle 5. Tienes que convertirlo a número.
- Si calculas un resultado `45.5`, a veces necesitas convertirlo a texto para enviarlo por red.

1. De String a Número (Parsing). Usamos las "Clases Envoltorio" (Wrapper Classes) de los primitivos: Integer, Double, etc.

De Tipo	A Tipo	Método
`String` "123"	`int` 123	`int x = Integer.parseInt("123");`
`String` "12.5"	`double` 12.5	`double y = Double.parseDouble("12.5");`

> **Peligro:** Si el texto no es un número válido (ej: "12a"), el programa lanzará una excepción **`NumberFormatException`** y se detendrá.

2. De Número a String. Hay dos trucos: la forma "elegante" y la forma "hacker".

Java
```
int numero = 500;

// Forma 1: Método oficial (Elegante y eficiente)
String s1 = String.valueOf(numero);

// Forma 2: Concatenación con vacío (Truco rápido)
String s2 = "" + numero;
```

C. Otras manipulaciones útiles

Además de pegar y convertir, estas son las 3 herramientas que usarás a diario:

1. Comparación de Contenido (`equals` vs `==`). Esta es la regla más importante de la unidad.

- **`==`**: Compara si son **el mismo objeto** en memoria (referencias).
- **`.equals()`**: Compara si tienen **las mismas letras** (contenido).

Java
```
String s1 = "Hola";
String s2 = new String("Hola");

if (s1 == s2) { ... }        // FALSE (Son objetos distintos en
memoria)
if (s1.equals(s2)) { ... }   // TRUE (Tienen las mismas letras)
```

Regla: Para comparar textos, **USA SIEMPRE `.equals()`**.

2. Extracción de trozos (`substring`). Permite recortar una parte de la cadena.

- `substring(inicio)`: Desde el inicio hasta el final.

- `substring(inicio, fin)`: Desde el inicio hasta el fin **(sin incluir el fin)**.

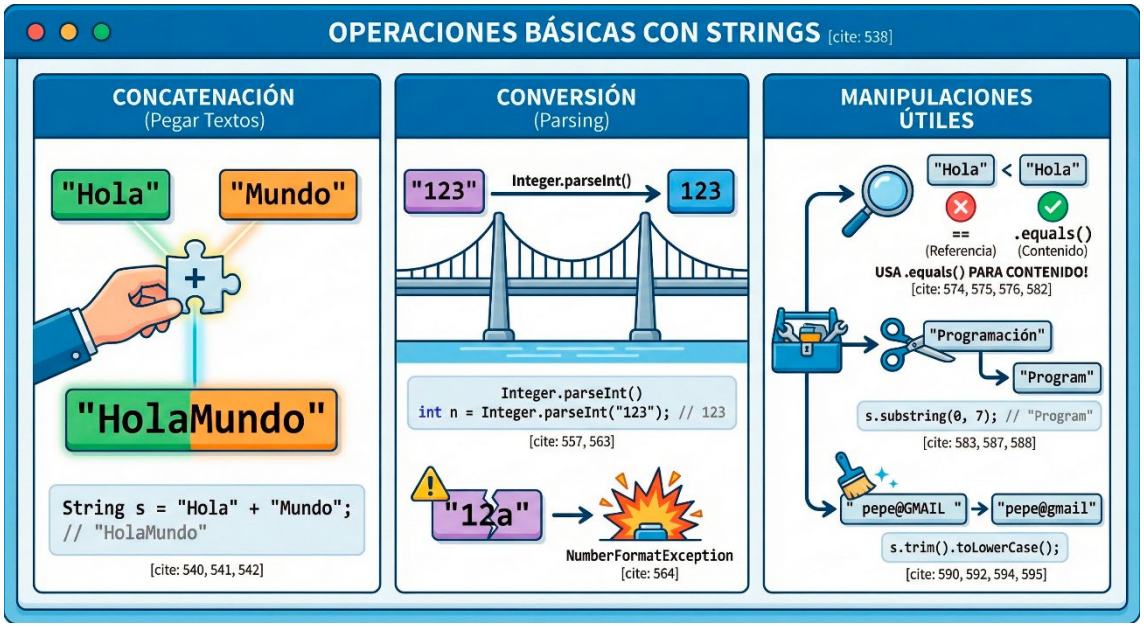
Java
```
String texto = "Programación";
String sub = texto.substring(0, 7); // De la 0 a la 6
System.out.println(sub); // "Program"
```

3. Limpieza y Formato (`trim`, `toUpperCase`). Muy útil para limpiar datos introducidos por usuarios desordenados.

Java
```
String input = "  pepe@GMAIL.com  ";

// 1. Quitar espacios sobrantes a los lados
input = input.trim(); // "pepe@GMAIL.com"

// 2. Convertir a minúsculas
input = input.toLowerCase(); // "pepe@gmail.com"
```



Resumen

1. Arrays Unidimensionales (Vectores)

Son objetos que almacenan una colección de datos del **mismo tipo** y tamaño **fijo**.

- **Ciclo de Vida:**

  1. **Declarar:** `int[] numeros;` (Crea la referencia en Stack. Valor `null`).
  2. **Crear:** `numeros = new int[5];` (Reserva memoria en Heap. Rellena con **0** o **null**).
  3. **Inicializar:** `numeros[0] = 10;` o usando atajo `{10, 20, 30}`.

- **Acceso:**

- Se usa el **Índice** entre corchetes `[]`.
- Empieza siempre en **0**.
- Termina en **`.length - 1`**.
- Error fatal si te sales: `ArrayIndexOutOfBoundsException`.

- **Propiedad Clave:** `.length` (sin paréntesis) nos dice el tamaño fijo.

2. Algoritmos con Arrays

No basta con guardar datos, hay que saber procesarlos.

- **Recorridos:**
  - **For Clásico:** `for (int i=0; i < a.length; i++)` -> Permite modificar y saber la posición.
  - **For-Each:** `for (int n : a)` -> Solo lectura, más elegante, no sabes la posición.

- **Búsquedas:**
  - **Lineal:** Lenta, pero funciona con datos desordenados.
  - **Binaria:** Muy rápida ("divide y vencerás"), pero requiere datos **ORDENADOS**.

- **Ordenación:**
  - **Académico:** Burbuja (intercambio con variable auxiliar).
  - **Profesional:** `Arrays.sort(array)` (Optimizado).

3. Arrays Multidimensionales (Matrices)

Tablas con filas y columnas. En realidad, son "arrays de arrays".

- **Sintaxis:** `int[][] matriz = new int[FILAS][COLUMNAS];`
- **Coordenadas:** `matriz[y][x]` o `matriz[fila][columna]`.
- **Recorrido:** Se necesitan **Bucles Anidados** (un `for` dentro de otro).
  - `matriz.length`: Número de filas.
  - `matriz[0].length`: Número de columnas de la fila 0.

4. Cadenas de Caracteres (String)

Objetos inmutables que representan texto.

- **Creación:**
  - `String s = "Hola";` (Recomendado. Usa el **Pool** de memoria para ahorrar).
  - `String s = new String("Hola");` (Fuerza un objeto nuevo duplicado).

- **Inmutabilidad:** Un String nunca cambia. Si concatenas (`+`), se crea uno nuevo y el viejo se descarta.
- **Comparación (¡VITAL!):**

- `==`: Compara si son el **mismo objeto** en memoria. (Mala idea para texto).
- `.equals()`: Compara si tienen las **mismas letras**. (Usa siempre esto).

- **Métodos útiles:**
  - `.length()`: Tamaño (¡Con paréntesis!).
  - `.charAt(i)`: Obtener letra.
  - `.substring(ini, fin)`: Recortar.
  - `Integer.parseInt(str)`: Convertir texto a número.

Tabla Array vs String

Esta tabla resume las confusiones más comunes en los exámenes:

Característica	**Array (int[])**	**String (String)**
¿Qué es?	Contenedor de datos.	Objeto de texto (inmutable).
Saber tamaño	`.length` (Propiedad)	`.length()` (Método)
Leer dato	`a[0]` (Corchetes)	`.charAt(0)` (Método)
Modificar	`a[0] = 5;` (Permitido)	**Imposible** (Hay que crear uno nuevo)
Comparar	`==` (Referencias)	`.equals()` (Contenido)